SOFTWARE ENGINEERING LAB - BCSE301P

Orbital Sentinel

Early-Warning Health Monitoring for Satellite Fleets
via Telemetry Anomaly Detection and Live Orbital Tracking

Study of SDLC Process Models, Model Selection &
IEEE 830 Software Requirements Specification

Name: Pushkar Kumar Mishra

Register Number: 24BPS1128

Programme: B.Tech CSE — Cyber Physical Systems

Institution: Vellore Institute of Technology, Chennai

Course: Software Engineering Lab – BCSE301P

Date: 14 July 2026

Contents

1. Problem Statement and Analysis
2. Study of SDLC Process Models (Waterfall, V-Model, Incremental, Prototype, Spiral, Agile)
3. Comparison of Process Models
4. Selected Process Model and Justification
5. Software Requirements Specification (IEEE 830 Standard Template)

1. Problem Statement and Analysis

Modern life quietly depends on satellites. Navigation (GPS and India's NavIC), weather forecasting, disaster response, broadcast, remote-area communications, and Earth observation for agriculture and climate all ride on fleets of spacecraft operated by agencies such as ISRO, NASA, and ESA. When one of these satellites begins to fail — a power channel sagging, a component overheating, a sensor drifting — the consequences ripple outward into the services people rely on, and the asset itself is worth hundreds of millions.

The difficulty is scale. A single modern spacecraft streams telemetry from thousands of channels, far more than a human operator can meaningfully watch in real time. Operations teams need automated help to catch the early signs of degradation before a fault becomes a failure. This is a genuine, active operational problem, and it is exactly the kind of monitoring that anomaly-detection research targets.

Orbital Sentinel is a proof-of-concept that addresses this problem in software. It learns the normal behaviour of a spacecraft's telemetry from historical data and flags abnormal behaviour as new readings arrive, explaining *which* sensor channel is responsible and ranking monitored satellites by risk so that operators' attention goes where it is most needed. A live 3D globe tracks real satellite positions and colours each craft by its health status, turning a wall of numbers into something an operator can read at a glance.

Because operational telemetry is private, the system is demonstrated on NASA's publicly released, expert-labelled telemetry from the SMAP satellite and the Curiosity Mars rover, while live positions come from public orbital-element data. The health monitoring is therefore shown on real recorded telemetry rather than a private live feed — an honest limitation that does not diminish the point, because the *technique* is what is being demonstrated, and it transfers almost unchanged to any sensor-driven infrastructure (power grids, industrial machinery, hospital monitoring, data centres).

1.1 Characteristics of the problem that drive model selection

- **A cleanly layered, separable architecture.** The system is a pipeline — data ingestion, anomaly detection, explanation and prioritisation, and visualisation — whose stages communicate through defined interfaces and can be built and tested independently.
- **Well-understood requirements.** The scope is bounded and can be written down in advance (this document does so), so requirements are not expected to churn heavily.
- **One genuine area of technical uncertainty.** It is not known up front which anomaly-detection model configuration performs best on the data; this benefits from a short experimental spike before the design is frozen.
- **A hard deadline and a required demonstration.** The system must be demonstrable by end of term, which strongly favours delivering a working core early and adding the visualisation as polish.
- **A largely single-developer effort**, which makes heavyweight team-oriented process ceremony unnecessary overhead.

2. Study of SDLC Process Models

The Software Development Life Cycle (SDLC) describes the stages a software product passes through from conception to retirement. A *process model* arranges those stages — how they are ordered, repeated, and overlapped. The six models required by the syllabus are studied below, each with its structure, strengths, weaknesses, and a short verdict on its fit to Orbital Sentinel.

2.1 Waterfall Model

The Waterfall model is the classical linear-sequential approach. Each phase is completed and signed off before the next begins, so the output of one phase flows down as the input to the next. It works best when requirements are completely known, stable, and unlikely to change.

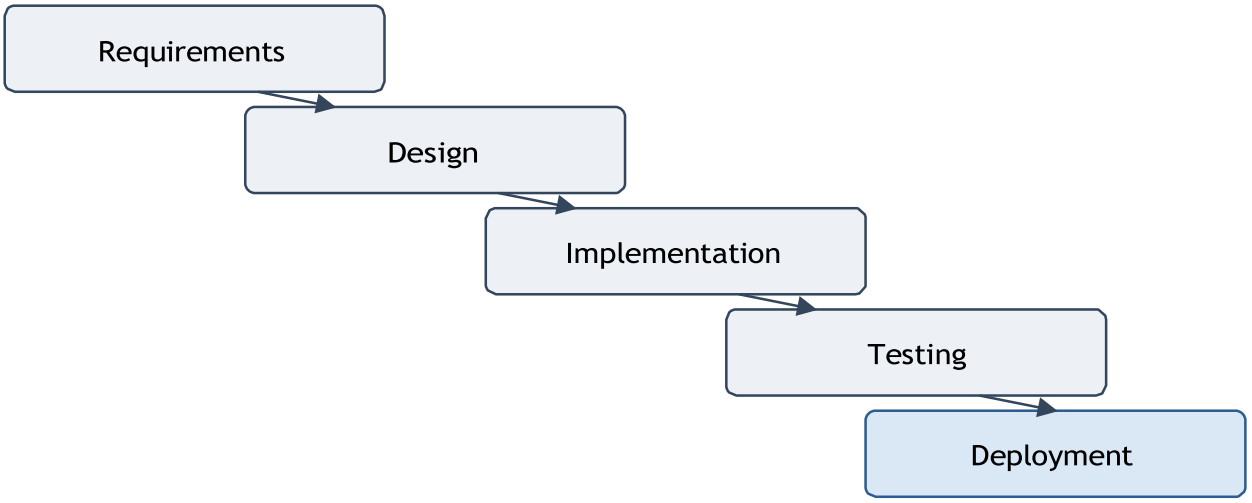


Figure 2.1 — The Waterfall model: strictly sequential phases.

Strengths: simple to understand and manage; clear milestones and documentation. **Weaknesses:** no working software until late; very poor at absorbing change; late-discovered defects are expensive. **Verdict:** a poor fit — it offers no early working demo and no room to iterate on the uncertain model-selection step.

2.2 V-Model (Verification and Validation)

The V-Model extends Waterfall by pairing each development phase on the left of the "V" with a corresponding testing phase on the right, with test planning happening early. It is favoured where reliability and traceable testing matter.

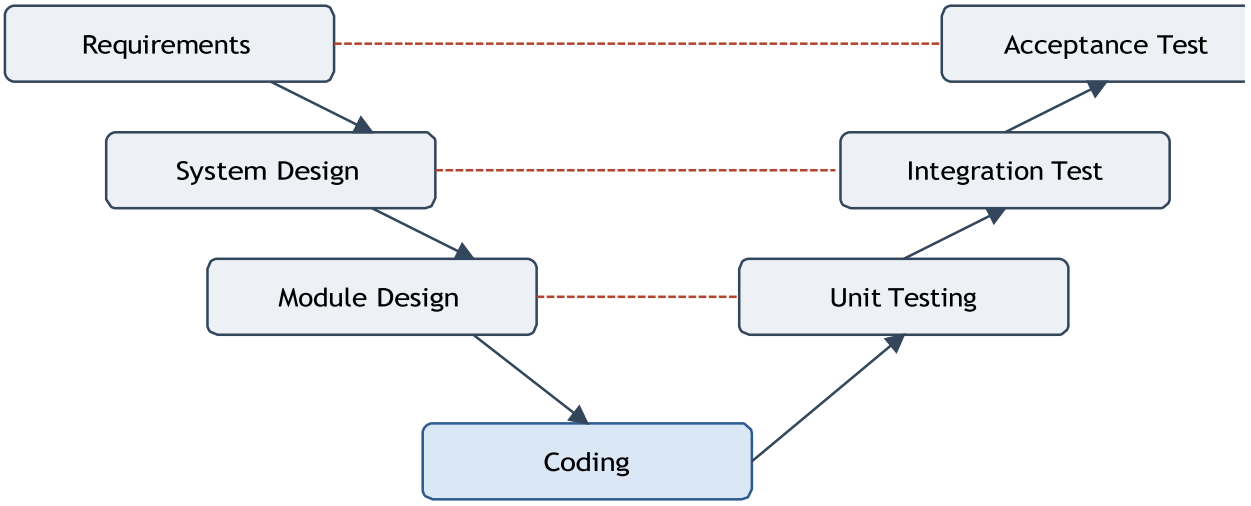


Figure 2.2 — The V-Model: each design phase (left) matched with a test phase (right).

Strengths: strong testing discipline; early defect detection; excellent traceability. **Weaknesses:** as rigid as Waterfall once underway; no early prototype. **Verdict:** its testing rigour is attractive and partly borrowed, but its rigidity cannot accommodate the model-selection iteration this project needs.

2.3 Incremental Model

The Incremental model breaks the system into a series of builds. Each increment runs its own mini design-code-test cycle and delivers a working slice of functionality; later increments add to the product until it is complete. Core features are built first, so a partially working system exists very early.

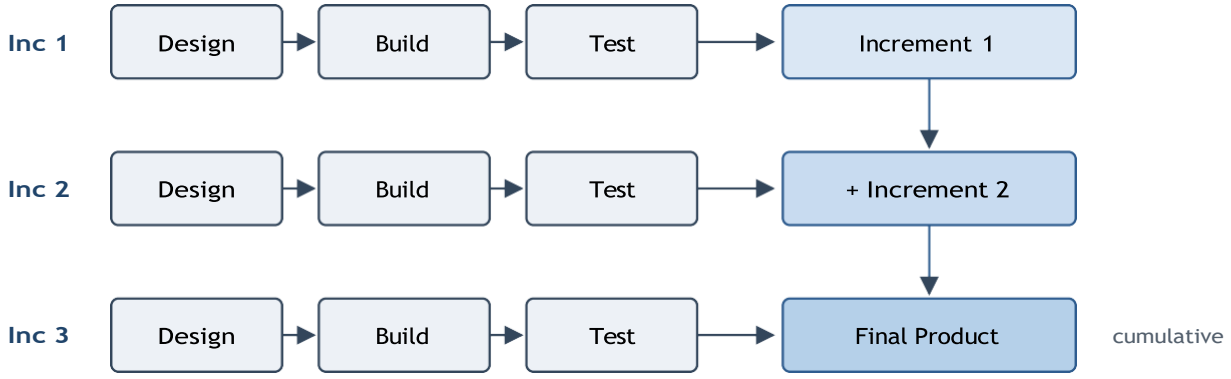


Figure 2.3 — The Incremental model: each increment is a working slice; the product grows build by build.

Strengths: a working system exists early; risk is spread across increments; easy to test in slices; ideal for systems that decompose cleanly. **Weaknesses:** needs a sound architecture

up front; poor decomposition causes rework. **Verdict:** a strong fit — the pipeline maps directly onto increments and each slice is independently demonstrable.

2.4 Prototype Model

In the Prototype model a quick, partial version is built early so that unclear requirements or unproven techniques can be explored before the real system is committed to. The prototype is refined through feedback, and may be evolved into the product or thrown away.

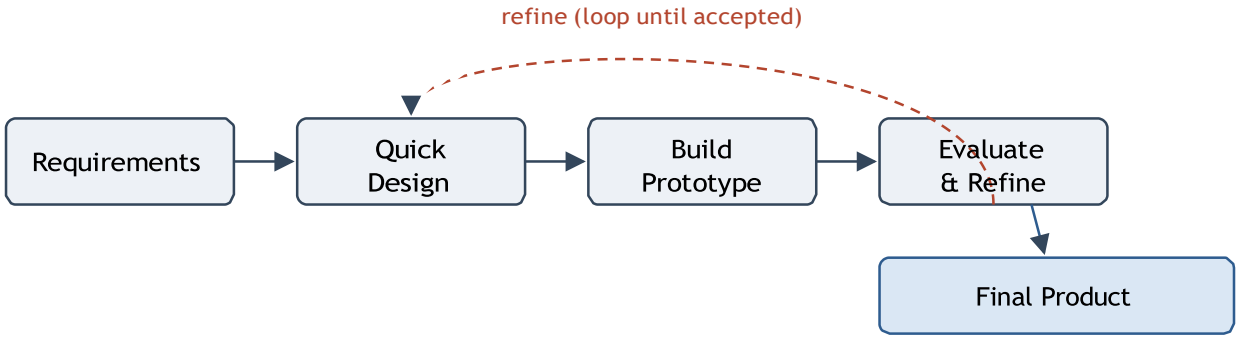


Figure 2.4 — The Prototype model: build, evaluate, and refine until requirements are validated.

Strengths: excellent for reducing uncertainty and clarifying vague requirements. **Weaknesses:** risk of throwaway effort; can invite endless tweaking; not a full delivery framework. **Verdict:** valuable specifically for the uncertain model-selection step, adopted as a short spike inside the incremental process rather than as the whole-project model.

2.5 Spiral Model

The Spiral model is risk-driven and iterative. Development proceeds through repeated loops, each divided into determining objectives, analysing risks, developing and testing, and planning the next iteration. It suits large, high-risk projects.

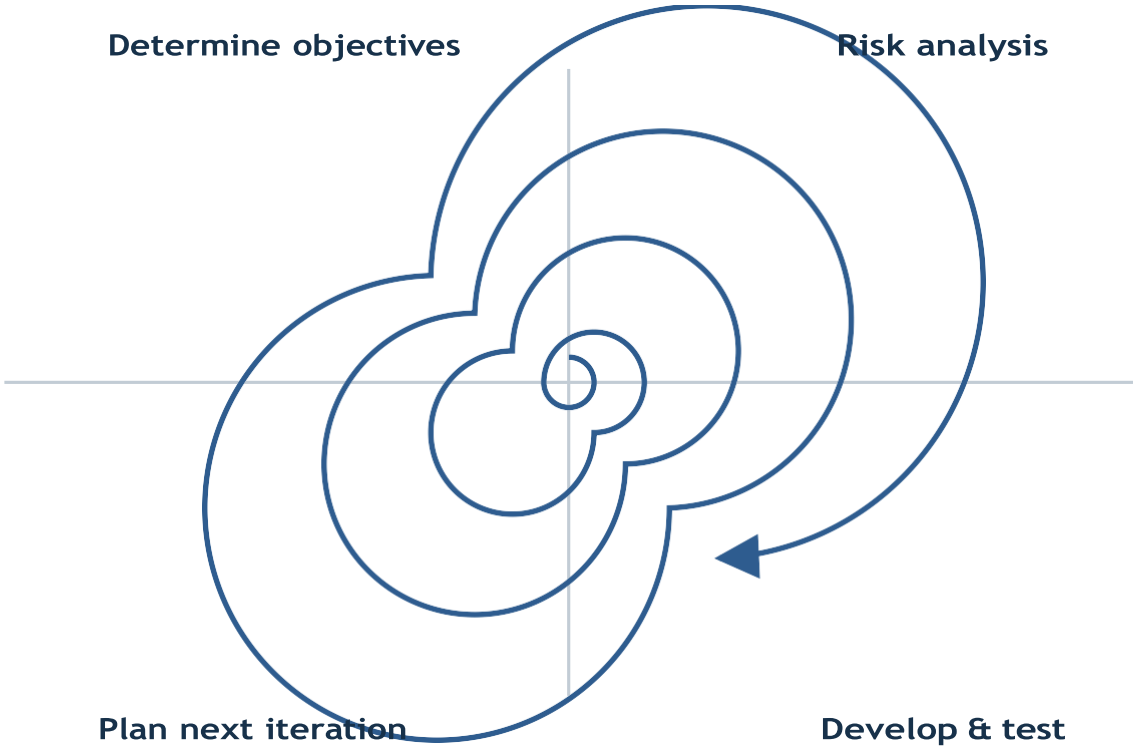


Figure 2.5 — The Spiral model: risk-driven loops through four quadrants of activity.

Strengths: continuous, explicit risk management; flexible; good for large evolving systems. **Weaknesses:** heavyweight; the formal per-cycle risk analysis is costly and needs expertise. **Verdict:** overkill — its formal risk process is disproportionate for a one-semester, single-developer project with a single well-understood risk.

2.6 Agile Model

Agile is iterative and incremental, valuing working software, close collaboration, and adaptability over heavy up-front planning. Work is delivered in short sprints, each producing a shippable increment, with frequent feedback and freedom to re-prioritise.

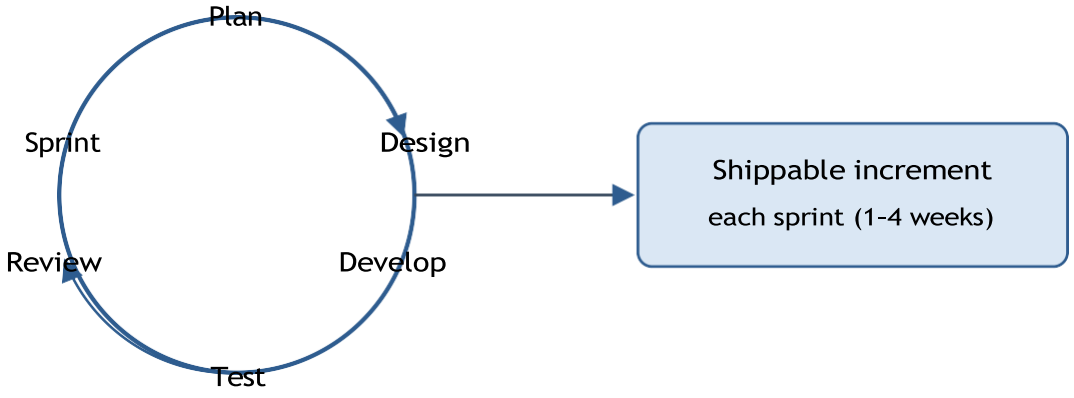


Figure 2.6 — The Agile model: repeated short sprints, each producing a shippable increment.

Strengths: highly adaptive to change; continuous feedback; frequent delivery. **Weaknesses:** needs active customer involvement and a team to pay off; light

documentation clashes with a course that grades a formal SRS; sprint ceremony adds overhead. **Verdict:** its iterative-delivery benefit is real, but its collaboration- and change-oriented machinery is unnecessary for a solo project with a fixed, graded specification.

3. Comparison of Process Models

Model	Requirements up front	Handles change	Early working demo	Risk handling	Overhead	Fit
Waterfall	All, fixed	Very poor	No (late)	Weak	Low	Poor
V-Model	All, fixed	Very poor	No (late)	Moderate	Low–Med	Poor
Incremental	Most, up front	Moderate	**Yes, early**	Good (spread)	Low–Med	**Strong**
Prototype	Partial / unclear	Good	Yes (throwaway)	Good (unknowns)	Medium	Partial (spike)
Spiral	Evolving	Good	Yes	**Excellent**	**High**	Overkill
Agile	Minimal, evolving	**Excellent**	**Yes, frequent**	Good	Med (ceremony)	Overhead-heavy

4. Selected Process Model and Justification

> **Selected model:** the **Incremental Model**, with a short throwaway **Prototype** spike for choosing the anomaly-detection model.

4.1 Why Incremental

1. **The architecture decomposes cleanly.** Orbital Sentinel is a pipeline whose stages talk through defined interfaces, so each stage is a natural increment that can be built and tested on its own.
2. **Requirements are known well enough to plan increments.** The scope is bounded and fully written in the SRS, so the increments can be laid out in advance — unlike Agile, which assumes requirements keep shifting.
3. **An early working demo is essential.** Incremental delivers a usable slice from the first increment, so a partial but demonstrable system exists well before the deadline. This also matches the developer's plan to build the detection pipeline first and treat the 3D globe as a later presentation layer.
4. **Risk is spread, not deferred.** Folding each stage in as it is completed surfaces integration problems early and cheaply, rather than at the end.
5. **Overhead stays proportionate.** For a single developer, Incremental gives useful structure without Agile's sprint ceremony or Spiral's formal per-cycle risk reviews.

4.2 Why a Prototype spike is folded in

The one thing Incremental does not settle by itself is the project's single real unknown: which anomaly-detection model configuration performs best on the telemetry. Rather than commit the whole design to one guess, a short throwaway prototype trains a couple of candidate detectors on a slice of the data and compares them. The winner is carried into the increments; the prototype is discarded. This uses the Prototype model exactly where it is strongest — resolving uncertainty — without letting it become the whole process.

4.3 Planned increments

Increment	Delivered capability	Demonstrates
0 — Spike	Candidate detectors compared on a data slice; best one chosen	Model-selection risk resolved (go/no-go)
1	Data ingestion + baseline anomaly detector, scored against labels	A working detection core with real accuracy numbers
2	Novelty layer: per-channel attribution + persistence/severity filter + risk ranking	The measurable contribution (fewer false alarms, explained alerts)
3	Live orbital tracking, 3D globe, alert dashboard	The complete, presentable system

Each increment is independently testable and demonstrable, aligning the process with both the course's emphasis on the software life cycle and the practical need for a reliable end-of-term demo.

PART 5

Software Requirements Specification

for

Orbital Sentinel

Version 1.0 approved

Prepared by Pushkar (24BPS1128)

Vellore Institute of Technology, Chennai

14 July 2026

Table of Contents

Revision History
1. Introduction
1.1 Purpose
1.2 Document Conventions
1.3 Intended Audience and Reading Suggestions
1.4 Product Scope
1.5 References
2. Overall Description
2.1 Product Perspective
2.2 Product Functions
2.3 User Classes and Characteristics
2.4 Operating Environment
2.5 Design and Implementation Constraints
2.6 User Documentation
2.7 Assumptions and Dependencies
3. External Interface Requirements
3.1 User Interfaces
3.2 Hardware Interfaces
3.3 Software Interfaces
3.4 Communications Interfaces
4. System Features
4.1 Telemetry Ingestion and Anomaly Detection
4.2 Anomaly Explanation and Risk Prioritisation
4.3 Live Orbital Tracking and 3D Visualisation
4.4 Alerting and Dashboard
5. Other Nonfunctional Requirements
5.1 Performance Requirements
5.2 Safety Requirements
5.3 Security Requirements
5.4 Software Quality Attributes
5.5 Business Rules
6. Other Requirements
Appendix A: Glossary
Appendix B: Analysis Models
Appendix C: To Be Determined List

Revision History

Name	Date	Reason For Changes	Version
Pushkar	14 July 2026	Initial version	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for **Orbital Sentinel**, version 1.0 — a proof-of-concept system that monitors the health of satellites by detecting anomalies in their telemetry, explaining and prioritising those anomalies, and presenting them alongside live orbital positions on a 3D globe. This SRS covers the complete proof-of-concept as demonstrated on publicly available telemetry and orbital-element data; it does not cover integration with any operator's private live telemetry feed, which is out of scope for this release.

1.2 Document Conventions

The keyword "shall" denotes a mandatory requirement and "should" a desirable one. Each functional requirement has a unique tag of the form `REQ-n`. Each system feature in Section 4 carries a priority of High, Medium, or Low; unless a contained requirement states otherwise, it inherits the priority of its feature. Physical quantities use SI units; monetary values, where they appear, are in Indian Rupees.

1.3 Intended Audience and Reading Suggestions

This document is intended for the developer, the course instructor and evaluators, and any teammates who may contribute to individual modules. The developer is the primary reader and should read the whole document. Evaluators may read Section 1 and Section 2 for context, then Section 4 for the detailed features. A reader new to the project is advised to begin with the Product Scope (1.4) and the Overall Description (Section 2) before proceeding to the specific requirements in Sections 3 and 4.

1.4 Product Scope

Satellites underpin navigation, weather forecasting, disaster response, communications, and Earth observation. A modern spacecraft reports telemetry from thousands of channels — more than any operator can watch by hand — so automated early warning of degradation protects both a costly asset and the public services that depend on it. Orbital Sentinel demonstrates such early-warning monitoring: it learns normal telemetry behaviour, flags abnormal behaviour as readings arrive, identifies the responsible sensor channel, and ranks monitored satellites by risk, with a live 3D globe for situational awareness. It is a proof-of-concept built on public data; its wider goal is to show a monitoring technique that transfers directly to any sensor-driven infrastructure, from power grids to industrial machinery.

1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

- K. Hundman et al., "Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding," ACM SIGKDD, 2018 — the baseline method and source of the SMAP/MSL dataset.
- "Explainable anomaly detection in spacecraft telemetry," Engineering Applications of Artificial Intelligence, 2024.
- "A Review of Anomaly Detection in Spacecraft Telemetry Data," MDPI Applied Sciences, 2025.
- "Transformer-based anomaly detection for satellite telemetry data," Acta Astronautica, 2025.
- "Deep Learning-Based Anomaly Detection in Spacecraft Telemetry on Edge Devices," 2026.
- NASA SMAP/MSL expert-labelled telemetry dataset (Telemanom release).
- CelesTrak — public Two-Line Element (TLE) orbital data; SGP4 / satellite.js propagation libraries.

2. Overall Description

2.1 Product Perspective

Orbital Sentinel is a new, self-contained system rather than a member of an existing product family. It is a cyber-physical monitoring system: it observes the sensor telemetry of a physical platform (a spacecraft) and infers that platform's physical health through computation. Architecturally it is a four-stage pipeline — data ingestion, anomaly detection, anomaly explanation and prioritisation, and visualisation — drawing on two independent public data sources: historical telemetry for health, and live orbital elements for position. The stages communicate through defined interfaces so that any stage can be developed and tested in isolation. A component diagram appears in Appendix B.

2.2 Product Functions

- Ingest spacecraft telemetry and replay it as a time-ordered stream.
- Detect anomalies in the telemetry against a learned model of normal behaviour.
- Attribute each anomaly to the responsible sensor channel(s).
- Suppress transient false alarms and assign a severity/risk level to each satellite.
- Fetch public orbital elements and compute live satellite positions.
- Render a 3D globe, colour satellites by risk, and allow click-to-inspect health.
- Raise a ranked alert list and log all detections for review and accuracy analysis.

2.3 User Classes and Characteristics

Analyst (Operator) — the primary and most frequent user; monitors the globe and alert list, inspects flagged satellites, and reads verdicts and explanations. Needs no knowledge of the internals. **Administrator** — configures which satellites are monitored, manages datasets, and tunes detection settings; typically the developer. **Evaluator** — the instructor or reviewer, who observes results and inspects logs and accuracy figures but does not operate the system.

2.4 Operating Environment

The system runs on a standard laptop or desktop (Windows, macOS, or Linux). The backend and machine-learning components use Python 3.10 or later with a standard ML framework (PyTorch or TensorFlow) and a local database. The dashboard and 3D globe are a web application viewed in a current version of Chrome, Firefox, or Edge. An internet connection is required only to fetch live orbital-element data; the telemetry analysis itself runs fully offline.

2.5 Design and Implementation Constraints

The health-monitoring data source is NASA's publicly released SMAP/MSL telemetry, which is a curated, static, historical dataset rather than a live feed; the system therefore replays recorded telemetry as a stand-in for a live health stream. Live orbital positions depend on public TLE data fetched over the internet. The system performs anomaly *detection* (flagging abnormal behaviour as it unfolds), not long-horizon *forecasting* of future failure. The anomaly-detection component is inherently probabilistic and is validated by accuracy metrics (precision, recall, false-positive rate) against the dataset's ground-truth labels rather than by deterministic pass/fail tests; the project explicitly requires a baseline-versus-enhanced comparison to establish the novelty. The whole system must run on a single commodity computer without specialised hardware.

2.6 User Documentation

The system is delivered with a README and quick-start guide covering installation, dataset setup, and how to run a monitoring session, together with brief in-application help (tooltips and an overview panel) explaining the globe controls, the meaning of risk colours, and how to read an anomaly explanation.

2.7 Assumptions and Dependencies

The system assumes that the public NASA telemetry dataset and public TLE feed remain available; that an internet connection is present when live positions are required; and that the chosen open-source libraries (the ML framework, the SGP4/satellite.js propagator, and the plotting/globe libraries) remain usable. It depends on the honest framing that positions are live while health telemetry is historical replay — a stated limitation rather than a defect. A change in dataset availability or library support would affect the implementation but not these requirements.

3. External Interface Requirements

3.1 User Interfaces

The application presents a 3D globe on which tracked satellites are shown at their live positions and coloured by risk level (for example green/amber/red). Selecting a satellite opens a health panel showing its current verdict, a confidence or severity value, the telemetry channel plots with any flagged anomalies highlighted, and the name of the channel that drove the most recent alert. A separate alert list ranks satellites by risk and supports drill-down into the corresponding health panel. All views fit a standard laptop screen without horizontal scrolling and display a clear busy indicator while a monitoring pass is running. Detailed visual design is left to a separate UI specification.

3.2 Hardware Interfaces

The system requires no specialised hardware and connects to no physical sensors; it is entirely dataset-driven. Its only hardware dependency is the standard computer on which it runs (processor, memory, display, and network interface). There are therefore no device-level control or data interactions to specify.

3.3 Software Interfaces

The system consumes public TLE orbital data from the CelesTrak service; the incoming data items are orbital element sets per satellite, used to compute positions via the SGP4 / satellite.js propagation library. The anomaly-detection component uses a standard machine-learning framework (PyTorch or TensorFlow) and reads the NASA telemetry dataset from local storage. A local database stores enrolled satellite configurations and detection logs. The backend and the web front-end exchange JSON messages: the backend emits, per satellite, the current risk level, verdict, confidence/severity, and the attributed channel; the front-end sends selection and configuration requests. Data shared across components (verdicts, risk levels, channel attributions) passes only through these defined JSON messages.

3.4 Communications Interfaces

Live orbital data is fetched over HTTPS from the public TLE provider. The front-end and back-end communicate over HTTP using JSON payloads. Each message carries a timestamp and a satellite identifier so that position and health information for the same satellite can be correlated. No email, encryption, or specialised transfer-rate requirements apply to this proof-of-concept beyond standard HTTPS for the external fetch.

4. System Features

4.1 Telemetry Ingestion and Anomaly Detection

4.1.1 Description and Priority

Priority: High. The core capability: ingest a satellite's telemetry and flag abnormal behaviour against a learned model of normal, scored against ground-truth labels.

4.1.2 Stimulus/Response Sequences

The Analyst starts a monitoring session for a satellite; the system streams that satellite's telemetry in time order, the detector scores each time window, and windows whose score exceeds the threshold are marked as anomalies and surfaced to the health panel and alert list.

4.1.3 Functional Requirements

- **REQ-1:** The system shall load the telemetry dataset and replay it as a time-ordered stream per satellite.
- **REQ-2:** The system shall normalise incoming telemetry and segment it into fixed-length time windows.
- **REQ-3:** The system shall score each window for abnormality using a trained baseline detector (an LSTM-based predictor with dynamic thresholding).
- **REQ-4:** The system shall label a window as anomalous when its score exceeds a configured threshold, and as normal otherwise.
- **REQ-5:** The system shall compare its labels against the dataset's ground-truth anomalies and compute precision, recall, and false-positive rate.
- **REQ-6:** On malformed or missing telemetry values, the system shall skip or interpolate the affected samples without terminating the session.

4.2 Anomaly Explanation and Risk Prioritisation

4.2.1 Description and Priority

Priority: High. This feature is the project's novelty: it adds an explanation and prioritisation layer over the baseline detector. It is additive — it always yields a usable output — so the contribution does not depend on beating a raw accuracy number, though it is expected to reduce false positives measurably.

4.2.2 Stimulus/Response Sequences

When an anomaly is detected, the system determines which channel(s) contributed most, applies a persistence check to discard isolated one-off spikes, assigns a severity level, and updates the satellite's position in the risk ranking.

4.2.3 Functional Requirements

- **REQ-7:** The system shall attribute each detected anomaly to the sensor channel(s) whose contribution to the anomaly score is greatest.

- **REQ-8:** The system shall apply a persistence filter that raises an alert only after a configurable number of consecutive anomalous windows, suppressing transient false alarms.
- **REQ-9:** The system shall assign each alert a severity level derived from the anomaly score and its persistence.
- **REQ-10:** The system shall maintain a risk ranking of all monitored satellites, ordered by current severity.
- **REQ-11:** The system shall report the false-positive rate with and without the persistence filter, so the improvement over the baseline can be quantified.

4.3 Live Orbital Tracking and 3D Visualisation

4.3.1 Description and Priority

Priority: Medium. The presentation layer: a live 3D globe showing real satellite positions, coloured by health, built after the detection core is working.

4.3.2 Stimulus/Response Sequences

The Analyst opens the globe view; the system fetches current orbital elements, computes positions, and renders each satellite; selecting one opens its health panel.

4.3.3 Functional Requirements

- **REQ-12:** The system shall fetch public TLE data for the monitored satellites and compute their current positions using an SGP4 propagator.
- **REQ-13:** The system shall render the satellites on a 3D globe and update their positions over time.
- **REQ-14:** The system shall colour each satellite according to its current risk level.
- **REQ-15:** The system shall open the corresponding health panel when a satellite is selected.

4.4 Alerting and Dashboard

4.4.1 Description and Priority

Priority: High. The dashboard that ties detection, explanation, and visualisation together and records results.

4.4.2 Stimulus/Response Sequences

As anomalies are detected, the alert list updates in risk order; the Analyst selects an alert to inspect the underlying telemetry and channel attribution; all detections are logged.

4.4.3 Functional Requirements

- **REQ-16:** The system shall present a risk-ordered alert list with each satellite's verdict, severity, and attributed channel.
- **REQ-17:** The system shall allow drill-down from any alert to the telemetry plots with the anomalous region highlighted.

- **REQ-18:** The system shall log every detection with its timestamp, score, attributed channel, severity, and outcome for later accuracy analysis.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The detector shall score a telemetry window and update the health panel within roughly one second of that window being ingested, so that replay feels responsive. The 3D globe shall maintain a smooth, interactive frame rate on a standard laptop for the demonstrated number of satellites. Fetching and propagating orbital elements for the monitored set shall complete within a few seconds of opening the globe view.

5.2 Safety Requirements

The system is software-only and controls no physical hardware, so it presents no direct safety hazard. It is explicitly a decision-support and demonstration tool: its verdicts must not be presented or used as the sole basis for any real spacecraft operational decision. This limitation shall be stated in the user documentation so the tool is not mistaken for an operational monitoring system.

5.3 Security Requirements

The system handles no personal data, and all external data it uses (telemetry dataset and orbital elements) is already public. It runs locally. Administrative functions (dataset management and detection configuration) shall be separated from the read-only analyst views so that a casual demonstration user cannot alter datasets or settings. No further authentication is required for this proof-of-concept.

5.4 Software Quality Attributes

Maintainability and modularity: the four pipeline stages shall interact only through their defined interfaces, so any stage can be replaced or tested independently using mock data. **Reliability:** malformed input shall be handled gracefully without crashing a session. **Portability:** the backend and dashboard shall run on any of the listed operating systems without code changes. **Testability:** because the telemetry dataset carries ground-truth labels, detection quality shall be measurable with standard accuracy metrics. **Usability:** an Analyst shall be able to interpret a satellite's health from the globe and health panel without training beyond the in-app help.

5.5 Business Rules

Analysts may view the globe, alerts, health panels, and logs, but may not change datasets or detection settings. Administrators may configure the monitored satellite set, manage datasets, and tune detection parameters. Evaluators may view all results and logs but perform no configuration. These roles imply the read-only versus administrative separation required in Section 5.3.

6. Other Requirements

Data requirements: the local database shall retain enrolled satellite configurations until deleted by an Administrator, and shall retain detection logs for at least the duration of the project for accuracy analysis. **Honest capability statement:** all documentation and demonstrations shall make clear that the system performs anomaly detection (not failure forecasting), that health telemetry is historical replay rather than a live operator feed, and that it is a proof-of-concept, not an operational tool. **Reuse:** the ingestion-to-evaluation pipeline is intended to be reusable, with only the dataset and feature definitions changing, so the same architecture can be applied to other sensor-driven monitoring domains.

Appendix A: Glossary

Telemetry — streams of measured values (voltages, temperatures, etc.) reported by a spacecraft over time. **TLE (Two-Line Element set)** — a public, regularly updated encoding of a satellite's orbit. **SGP4** — the standard model that turns a TLE into a position at a given time. **Anomaly** — telemetry behaviour that departs from the learned normal. **LSTM** — a recurrent neural network suited to sequential data, used as the baseline detector. **Channel attribution** — identifying which sensor channel is responsible for a detected anomaly. **Persistence filter** — a rule that raises an alert only after several consecutive anomalous windows, reducing false alarms. **Precision / Recall / False-positive rate** — standard accuracy metrics measured against ground-truth labels. **SMAP / MSL** — the two NASA missions whose labelled telemetry forms the public benchmark dataset.

Appendix B: Analysis Models

B.1 System Architecture

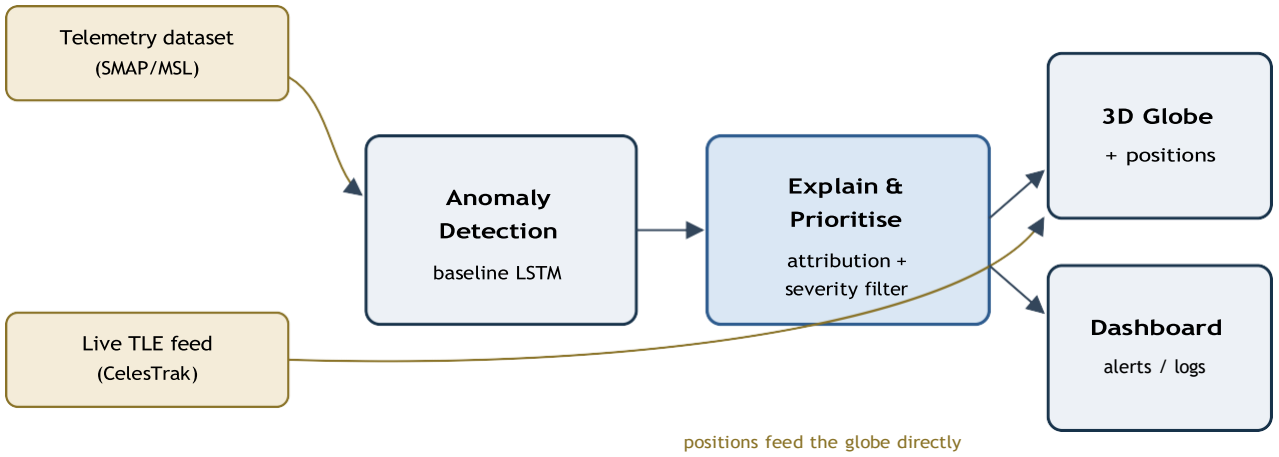


Figure B.1 — Component architecture. Positions (live TLE) feed the globe; health (replayed telemetry) flows through detection and prioritisation.

B.2 Data Flow Diagram — Level 0 (Context)

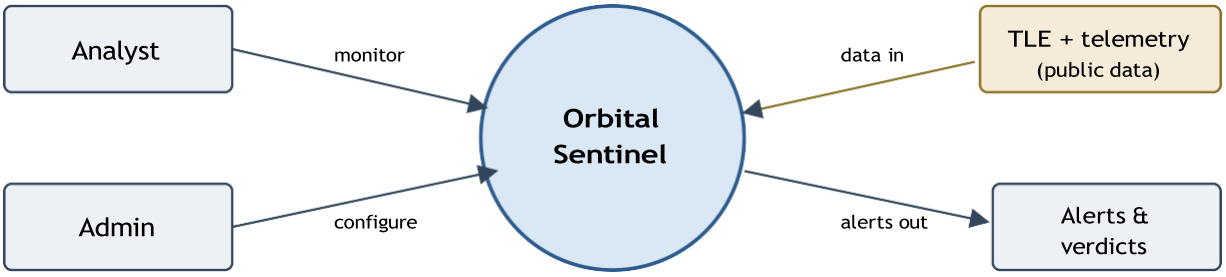


Figure B.2 — Context diagram: actors, the system, its public data sources, and its outputs.

B.3 Data Flow Diagram — Level 1

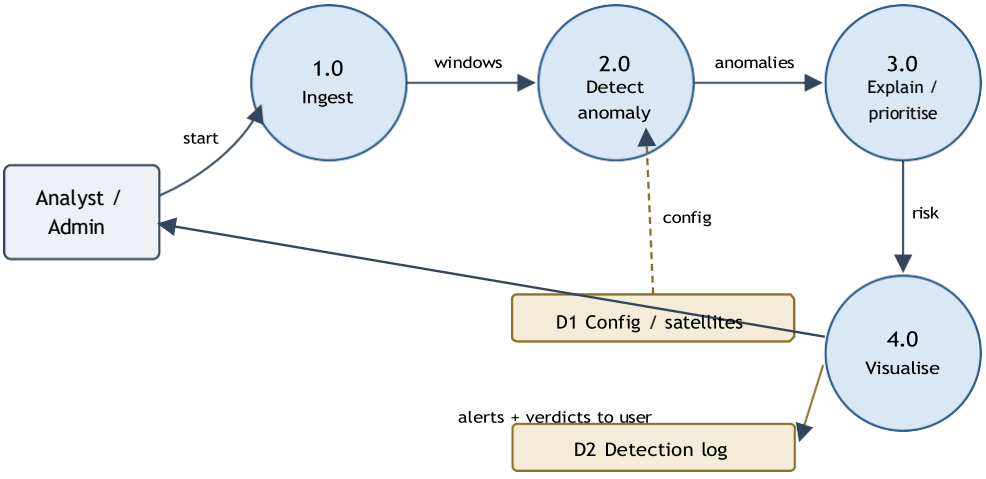


Figure B.3 — Level-1 DFD: ingest, detect, explain/prioritise, visualise, with the configuration and log stores.

B.4 Use Case Diagram

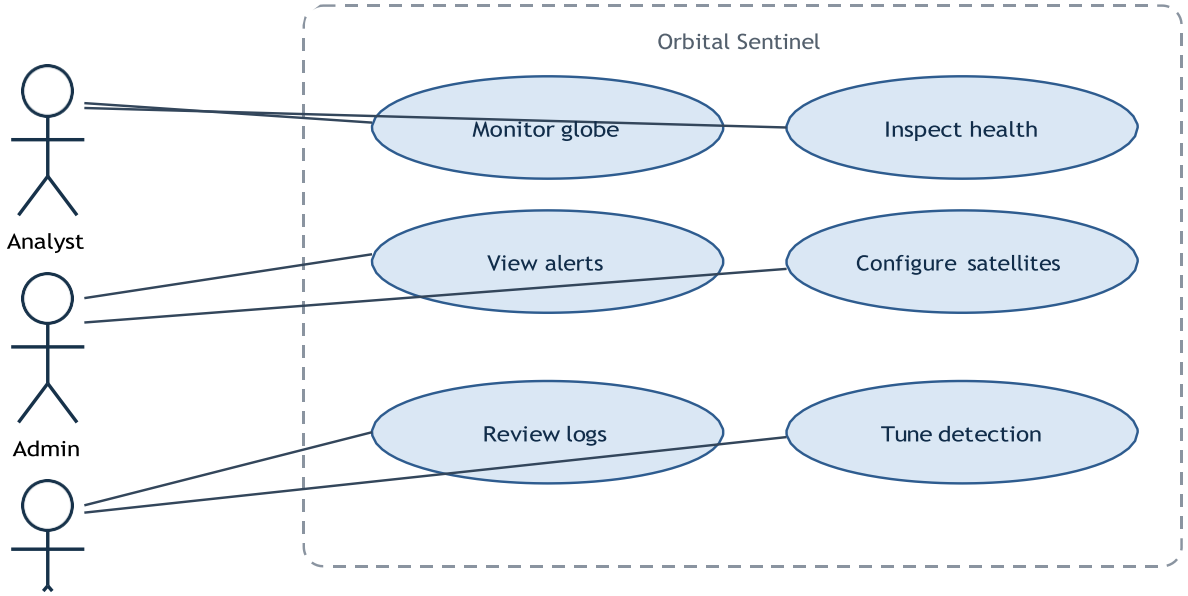


Figure B.4 — Use cases and actors. Analyst monitors and inspects; Admin configures and tunes; Evaluator reviews logs.

Appendix C: To Be Determined List

1. **TBD-1:** the exact anomaly-score threshold and the number of consecutive windows used by the persistence filter, to be fixed empirically during Increment 2.
2. **TBD-2:** the final set of satellites shown on the live globe, to be chosen from those available in the public TLE feed.
3. **TBD-3:** the specific channel-attribution method (reconstruction-error ranking versus a SHAP-style attribution), to be selected after the Increment 0 spike.